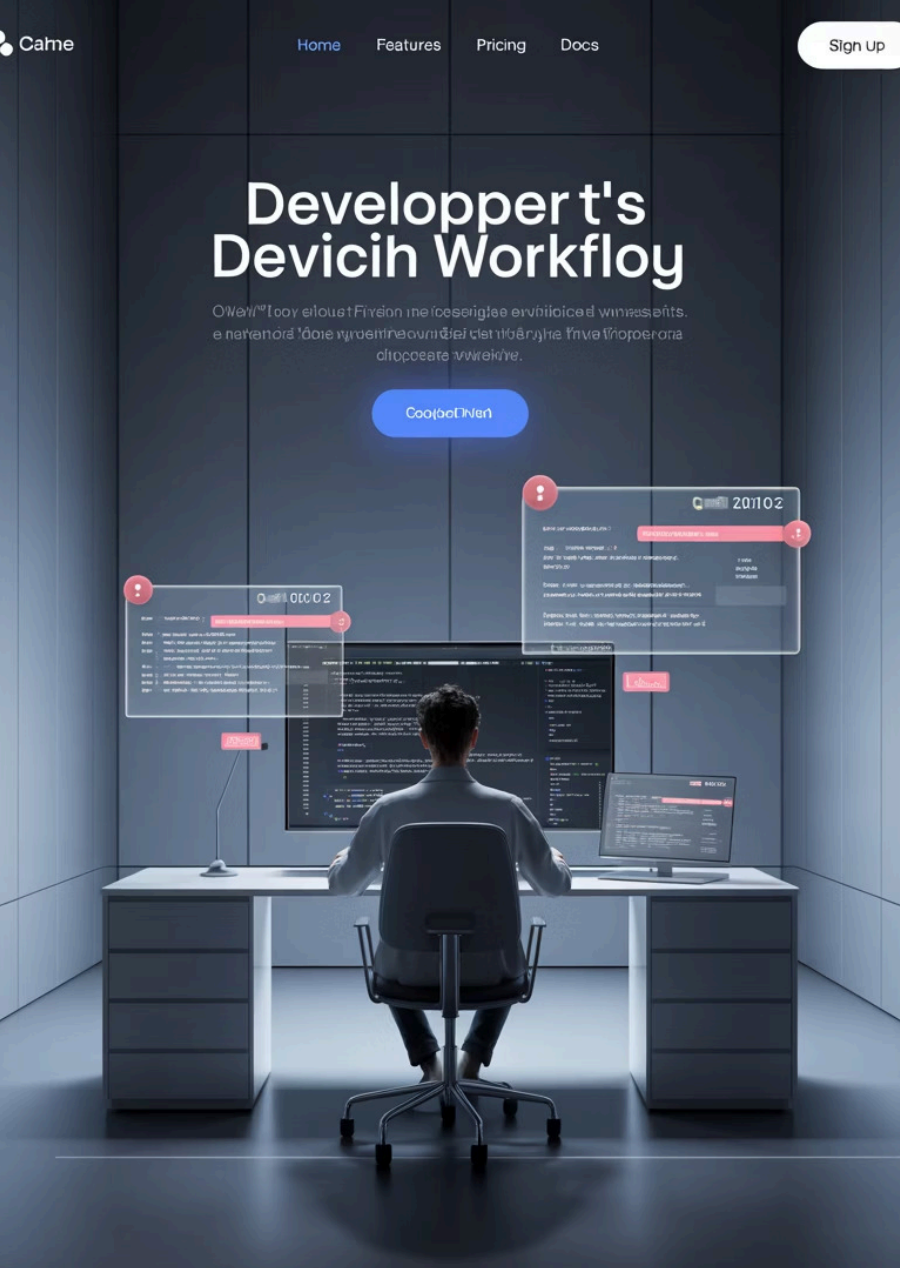




Validação de Formulários com React Hook Form e Yup

Nesta apresentação, vamos mergulhar em uma das partes mais importantes de qualquer aplicação web: a validação de formulários. Usando o poder do React Hook Form combinado com o Yup, você aprenderá como implementar validações robustas, escaláveis e com ótima experiência para o usuário.

Ideal para desenvolvedores iniciantes em React que desejam elevar a qualidade de seus formulários.

O que é Validação por Esquema?

A validação por esquema (schema validation) é uma abordagem **declarativa** para definir regras de validação de dados em formulários.

Em vez de escrever múltiplas condições para verificar cada campo manualmente, você cria um "**esquema**" - um molde que descreve exatamente como os dados devem ser estruturados.

1 Benefícios

- Código mais limpo e organizado
- Regras centralizadas e reutilizáveis
- Fácil manutenção e escalabilidade

2 O Yup permite definir

- Campos obrigatórios
- Tipos de dados esperados
- Regras específicas (min/max, formato)
- Mensagens de erro personalizadas



"Um esquema de validação é como um contrato entre seu formulário e os dados que ele deve receber."

Integrando Yup com React Hook Form

1. Instalação das Dependências

```
npm install @hookform/resolvers yup
```

O pacote `@hookform/resolvers` é o responsável por fazer a ponte entre o React Hook Form e o Yup.

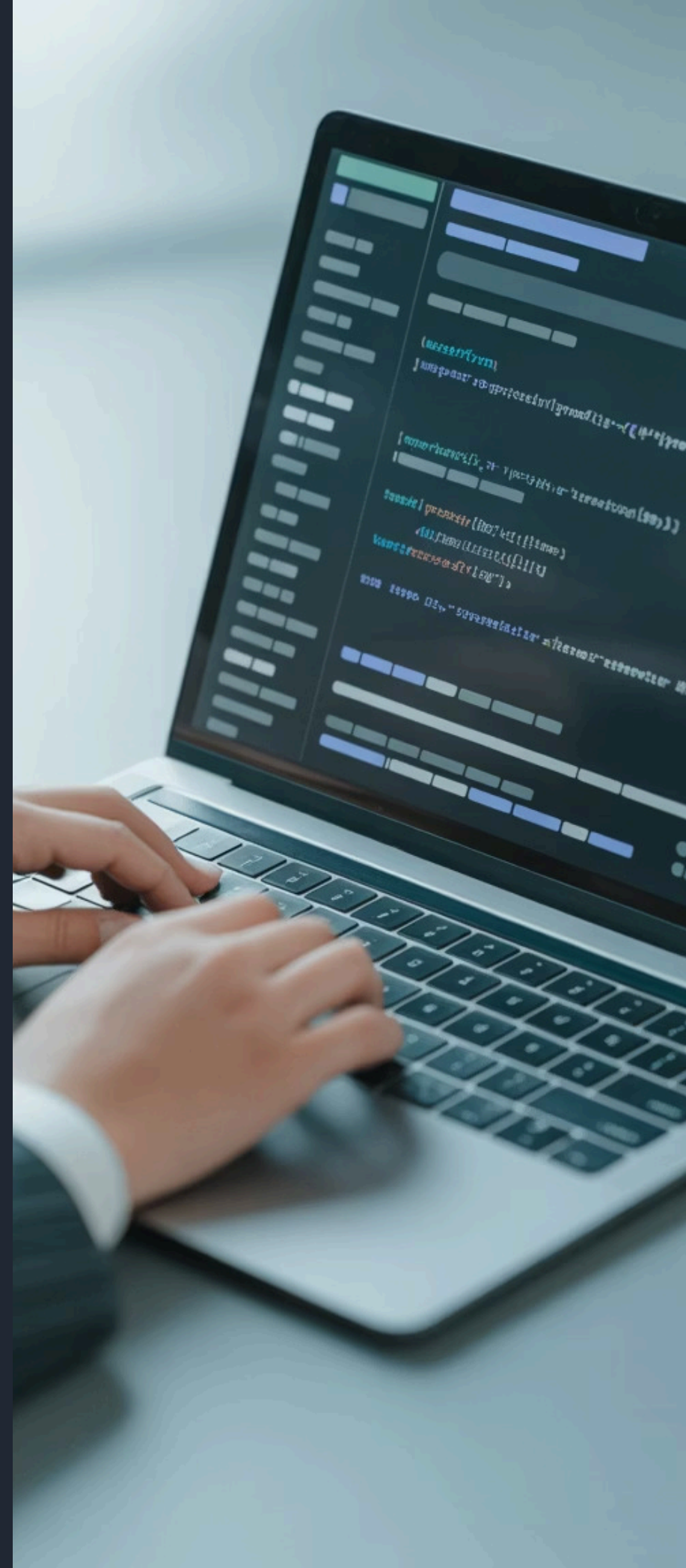
2. Definição do Esquema com Yup

```
const esquemaValidacao = yup.object({
  nome: yup.string().required('Nome é obrigatório'),
  email: yup.string()
    .email('Formato de e-mail inválido')
    .required('E-mail é obrigatório'),
  idade: yup.number()
    .positive('Idade deve ser positiva')
    .integer('Idade deve ser um número inteiro')
    .required('Idade é obrigatória')
});
```

3. Configuração no React Hook Form

```
const { register, handleSubmit, formState: { errors } } =
  useForm({
    resolver: yupResolver(esquemaValidacao)
  });
```

Com essa configuração, o React Hook Form usará o Yup para validar os dados do formulário automaticamente.



Exibindo Erros e Dicas Práticas

Exibindo Mensagens de Erro

```
<input {...register("email")} />
{errors.email &&
  <span className="erro">
    {errors.email.message}
  </span>}
```

Validação em Tempo Real

Para validar enquanto o usuário digita:

```
useForm({
  mode: 'onChange', // ou 'onBlur'
  resolver: yupResolver(esquemaValidacao)
});
```

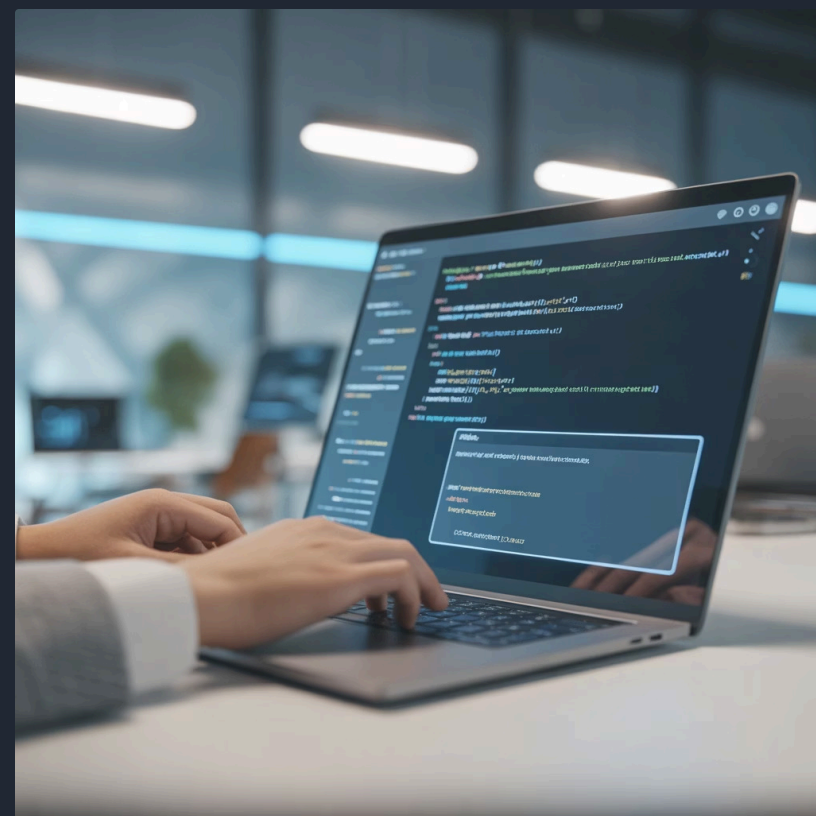
Validações Condicionais

Use `.when()` para validações que dependem de outros campos:

```
senha2: yup.string()
  .when('senha', {
    is: (val) => val?.length > 0,
    then: (schema) => schema
      .required('Confirme sua senha')
      .oneOf([yup.ref('senha'),
        'As senhas não conferem'])
  })
```

Dicas para Formulários Profissionais

- Sempre forneça **feedback visual** claro
- Use **validação assíncrona** para verificações com o servidor
- Implemente **testes** para seus esquemas de validação
- Separe esquemas complexos em módulos reutilizáveis



Com estas técnicas, seus formulários ficarão mais robustos, confiáveis e proporcionarão uma experiência muito melhor para seus usuários!